Informe Preliminar : Desafío 2

Materia : Informática II **Semestre 2026-2**

Integrantes : **Santiago Osorio Fernandez**
Juan José Guarin Atehortua

Universidad de Antioquia

Contextualización

En el presente desafío se propone diseñar una solución lógica con programación orientada a objetos para una red social que se pueda escalar a nivel nacional en Colombia. Se propone una solución que será probada para 1000 usuarios en la fase inicial pero que podrá ser escalada a millones de usuarios. La solución planteada tendrá una clase que se encargará de crear las listas necesarias para los almacenamientos de los diferentes tipos de datos de la red social, esto nos ayudará al almacenamiento secuencial de los datos y así no depender de librerías externas como Vector o list.

La base principal serán los usuarios, en torno a ellos se crearán las clases complementarias : Mensaje, Album, Fotos. Asimismo la clase Red social que define las reglas de negocio globales de la solución dadas en el enunciado y que a su vez se encarga de las conexiones entre amigos que se manejan con una matriz de adyacencia que al mismo tiempo permitirá determinar las sugerencias de amigos.

Para verificar el funcionamiento de la solución se programará un generador de datos heterogéneos con casos extremos para asegurarnos de que la solución funcione correctamente, garantizando la escalabilidad del proyecto cuando se disponga de la infraestructura física requerida (servidores y almacenamiento a gran escala). Finalmente, se plantea una clase que rastrea los recursos utilizados la cual será transversal y tiene un rol de auditar cada funcionalidad en su consumo de bytes e iteraciones dentro del programa luego de cada ejecución.

Análisis y diseño :

En esta sección se detallan las clases planteadas para el diseño de la solución, especificando sus atributos, métodos y responsabilidades dentro del sistema. Se presenta la explicación del diagrama de clase que acompaña este reporte, el cual condensa los componentes esenciales identificados hasta el momento que creemos necesarios para desarrollar el desafío. Es importante recordar que en la etapa de desarrollo se podrá modificar las clases según las necesidades o situaciones que vayamos observando en el camino y que no se hayan previsto hasta este momento.

1. **Nodo<T> y ListaDinamica<T>** (Nuestra lista sin STL) **Responsable : JUAN**

Ante la imposibilidad de utilizar la libreria STL, creamos nuestra propia lista enlazada a la memoria dinamica, estos son los metodos de la clase. El Nodo : Es la cajita básica en RAM. Guarda el dato y el puntero.

- **agregarAlFinal(elem)**: Pone un dato nuevo al final.
- **obtener(pos)** / **eliminar(pos)**: Lee o borra un dato reacomodando punteros con delete.
- **obtenerTamano()**: Devuelve cuántos datos hay guardados.

2. **Usuario** (Perfil del usuario) **Responsable : JUAN**

Guarda la información del perfil y sus contactos. Usamos listas de punteros (Usuario*, Album*, Mensaje*) para no duplicar objetos pesados en memoria. Atributos : Username, correo, fecha de nacimiento, ciudad, país, foto y fecha de inscripción. Listas relacionales: Amigos, solicitudes pendientes, álbumes y sus mensajes enviados/recibidos.

- Guarda los datos personales y sus listas de contactos (amigos, solicitudes, álbumes, mensajes).
- **Usuario(copia)**: Duplica bien los datos y punteros en memoria.
- **operator== / !=**: Compara usuarios por su Username.

3. **RedSocial** (El cerebro y la matriz de amistades) **Responsable : JUAN**

Clase RedSocial Maneja a todos los usuarios y las conexiones de la red: Guarda la lista global de usuarios y la matriz de adyacencia `bool**` en memoria dinámica para revisar amistades al instante.

- **cargarPersistencia()**: Lee los `.txt` al arrancar el programa para llenar la matriz y los usuarios.
- **registrarAmistad()**: Pone en `true` la casilla correspondiente en la matriz de adyacencia.
- **obtenerMejoresAmigosSorteados()**: Revisa amigos en común en la matriz, asigna las probabilidades ($1\times$, $2.5\times$, $3.5\times$) y saca 3 fotos para el inicio de sesión.
- **obtenerSugerenciasK()**: Cuenta cuántos amigos en común tiene un usuario con desconocidos en la matriz, los ordena de mayor a menor y muestra los K mejores.

4. **Album y Foto: Galería** de la red social **Responsable : SANTIAGO**

El almacenamiento del contenido multimedia se realizará mediante la creación de rutas absolutas estandarizadas de Linux las cuales simularán una ruta de almacenamiento tanto para los álbumes como para las fotos. Como atributos de la clase álbum se definirán el nombre, la ruta de acceso, privacidad y una lista de

fotos, para la clase Fotos se definen los atributos fecha de captura, tamaño de bytes, ruta de almacenamiento y los likes de la foto. como métodos se tendrán los constructores y getters/setters. Para este ejercicio solo mostraremos estas rutas y no los archivos de imágenes reales. Para asegurar el carrusel circular se trabajará con los índices de la lista y punteros auxiliares para la navegación (Anterior/Siguiente).

5. **Mensaje y Gestor Mensajes** (Chat) **Responsable : SANTIAGO**

Esta clase está pensada para almacenar las conversaciones entre los usuarios como texto plano, almacenará como atributos los datos del emisor, receptor, mensaje, estado de lectura y los datos de tiempo que serán utilizados para la búsqueda de mensajes. El método de filtrado se diseñará recorriendo la lista secuencial y validando si la fecha del mensaje está dentro del rango de búsqueda. Los métodos que se crearán para esta clase son:

- **enviarMensaje()**: Guarda el texto (hasta 512 caracteres).
- **obtenerNuevos()**: Muestra los mensajes no leídos ordenados por fecha y los marca como leídos.
- **buscarPorFecha()**: Filtra mensajes entre dos fechas.

6. **ResourceTracker** (Medición de rendimiento) **Responsable : SANTIAGO**

Esta clase es la que se encargará de auditar y rastrear el consumo de memoria de todas las funcionalidades del programa. Se analizará la memoria RAM mediante contadores y variables acumulativas. Esto nos permitirá tener el conteo de las iteraciones y de la memoria utilizada por los arreglos dinámicos en cada operación.

- **registrarIteraciones()**: Suma las vueltas de los bucles.
- **calcularMemoriaActual()**: Mide la memoria exacta ocupada en RAM (Bytes/KB).

7. **SistemaUdeABook** (Menú de consola) **Responsable : SANTIAGO**

Esta será la interfaz de nuestro programa, se plantea una solución en consola para las pruebas piloto; si la solución es eficaz y se decide desarrollar la segunda etapa, se procedería a la creación de la interfaz gráfica del sistema. Aquí iniciaremos el programa y le daremos las opciones de interacción al usuario. En este módulo se llamarán todas las clases y listas necesarias a las diferentes funcionalidades, lo que nos asegura separar la lógica del negocio de la presentación visual del programa.

Diagrama de Gantt para el desafío

Tarea / Módulo de Desarrollo	Responsable	05/10/26	06/10/26	07/10/26	08/10/26	09/10/26	10/10/26	11/10/26	12/10/26	13/10/26	14/10/26	15/10/26	16/10/26
Definición Formato Archivos Planos	Ambos	X	X										
Análisis & Modelado (Usuario, RedSocial)	Juan José	X	X	X									
Plantilla ListaDinamica (Memoria Dinámica)	Juan José			X	x	x							
Clase Usuario	Juan José					x	x	x					
Clase RedSocial & Matriz Adyacencia	Juan José							x	x	x			
Algoritmos Prioridad (Fotos) & K Sugerencias	Juan José									x	x		
Persistencia Usuarios & Grafo Amistades	Juan José										x	x	
Análisis & Modelado (Mensajes, Galería)	Santiago	X	X	X									
ResourceTracker (Iteraciones & Memoria Bytes/KB)	Santiago			X	x	x							
Clases Foto, Album & GestorGaleria (Carrusel)	Santiago					x	x	x	x				
Clase Mensaje & GestorMensajes (Fechas/Filtros)	Santiago							x	x	x			
SistemaUdeABook (Menú Consola Interactivo)	Santiago									x	x		
Persistencia Mensajes, Álbumes & Fotos	Santiago										x	x	
Entrega Informe preliminar	Ambos					x							
Integración	Ambos									x	x	x	
Pruebas con Dataset 1000 Usuarios & Optimización	Ambos										x	x	
Video YouTube (5-15 min) & Informe Final	Ambos											x	x
Entrega Final Código, Video & Sustentación	Ambos												x

Diagrama de clases

